Phase 2

1. Introduction

Phase 1 defined the security requirements, threat model, STRIDE analysis, PASTA analysis, risk matrix, and framework mapping for the system. Phase 2 converts those requirements and threats into a practical secure architecture blueprint.

The Secure Inventory Management Platform is designed as a secure cloud-native inventory application that supports authentication, role-based access control, inventory item management, stock updates, supplier and order record handling, reporting, audit logging, security monitoring, and event-driven activity tracking. The architecture is designed to protect business-critical inventory data from unauthorized access, tampering, information disclosure, privilege escalation, denial of service, and repudiation.

The platform follows security-by-design principles. Every major component is placed inside a controlled architecture where users, APIs, databases, cache services, event streaming, containers, and Kubernetes resources are protected through authentication, authorization, network isolation, secure configuration, logging, and monitoring.

2. Architecture Objectives

The main objective of this secure architecture is to design a protected, scalable, and container-ready inventory management platform. The architecture focuses on confidentiality, integrity, availability, accountability, and secure deployment.

The specific architecture objectives are:

1. To design a secure web application architecture for inventory users, staff, administrators, managers, and auditors.
2. To enforce authentication and authorization before allowing access to protected inventory APIs.
3. To apply role-based access control so that Admin and Staff users can only perform actions allowed by their roles.
4. To protect inventory records, user accounts, audit logs, and security alerts from unauthorized access or modification.
5. To use Docker containers for packaging the application and supporting services in an isolated and repeatable environment.

6. To design Kubernetes deployment with separate services, secrets, resource controls, and network restrictions.
7. To include Redis for secure failed-login tracking and caching-related support.
8. To include Kafka for event-driven security and inventory activity publishing.
9. To maintain audit logs for login attempts, product creation, product updates, product deletion, unauthorized access, and suspicious activity.
10. To support Zero Trust principles where no user, request, service, or container is trusted automatically.
11. To map the architecture with SABSA layers so that business requirements, security controls, logical design, physical deployment, and operational security are connected.

3. Scope of Phase 2

The scope of Phase 2 is secure architecture design. This phase focuses on explaining how the system should be structured securely before and during implementation.

The Phase 2 architecture covers the following areas:

- Web frontend architecture
- Backend API architecture
- Authentication and authorization design
- MongoDB database architecture
- Redis cache and failed-login tracking design
- Kafka event streaming design
- Audit logging and security alert design
- Docker container architecture
- Kubernetes deployment architecture
- Network segmentation and trust boundaries
- Zero Trust security model
- SABSA architecture mapping
- Security control placement across the system

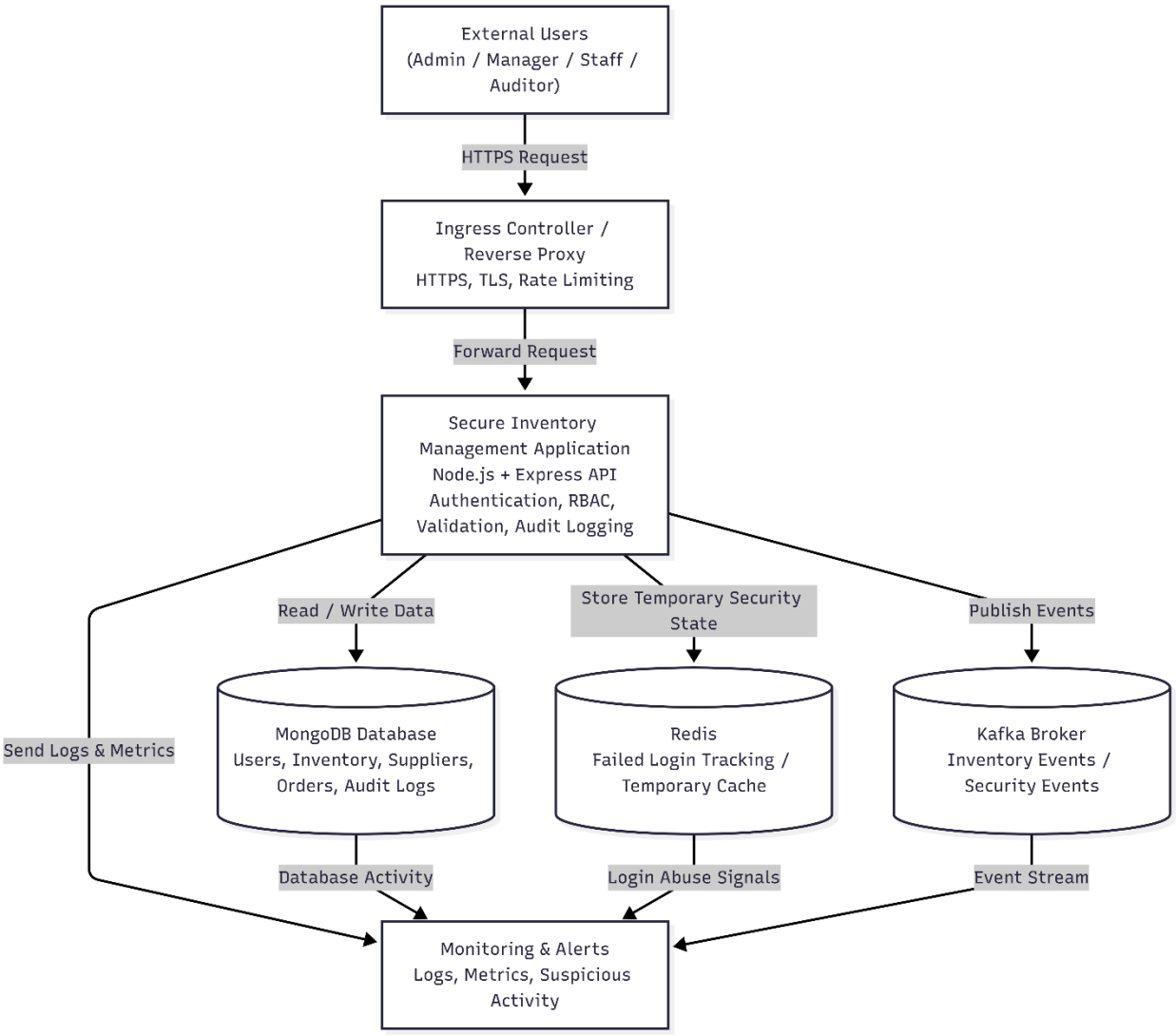
4. High-Level Secure Architecture

The Secure Inventory Management Platform is designed as a secure cloud-native web application. The architecture separates the user interface, backend API, database, cache, event streaming, audit logging, and monitoring components. This separation improves security because each component has a clear responsibility and can be protected using independent controls.

At a high level, users access the system through a browser over HTTPS. The request first reaches the public entry point, such as an Ingress Controller or reverse proxy. From there, requests are forwarded to the application container running the backend API. The backend API performs authentication, authorization, input validation, business logic processing, audit logging, and secure communication with internal services.

The database is not directly exposed to external users. It is placed inside an internal network boundary and can only be accessed by the backend API. Redis is used for failed-login tracking, caching, and security-related temporary state. Kafka is used for event-driven activity publishing, such as product creation, product updates, product deletion, login activity, and security events. Audit logs and security alerts are generated to support accountability, monitoring, and incident response.

Diagram



4.2 Architecture Explanation

The architecture uses a layered security approach. The first layer is the user access layer, where only authenticated users can access protected platform features. The second layer is the application layer, where backend authorization checks decide whether a user can perform an action. The third layer is the data layer, where MongoDB stores users, inventory records, supplier records, orders, audit logs, and security information. The fourth layer is the supporting services layer, where Redis and Kafka support security tracking and event-driven architecture. The final layer is the monitoring layer, where logs, failed login attempts, suspicious behavior, and important inventory events are observed.

This architecture avoids direct access from external users to sensitive backend services. Users cannot directly connect to MongoDB, Redis, or Kafka. All access must pass through

the backend API, where authentication, authorization, validation, logging, and monitoring are applied.

4.3 Main Architecture Layers

Layer	Component	Purpose	Security Control
User Layer	Admin, Inventory Manager, Staff/User, Auditor	Access platform features through browser	HTTPS, login, role-based access
Entry Layer	Ingress Controller / Reverse Proxy	Routes traffic to backend service	TLS, rate limiting, secure headers
Application Layer	Node.js / Express API	Handles authentication, authorization, inventory logic, supplier logic, and reports	JWT/session validation, RBAC, input validation
Data Layer	MongoDB	Stores users, inventory, suppliers, orders, audit logs, and security alerts	Internal access only, secrets, backups
Cache Layer	Redis	Tracks failed login attempts and temporary security state	Internal service access, restricted network
Event Layer	Kafka	Publishes inventory and security events	Internal broker access, event audit trail
Monitoring Layer	Audit logs, metrics, alerts	Detects failed logins, suspicious actions, and system issues	Logging, alerting, incident support

4.4 Security Value of This Architecture

This architecture improves security in the following ways:

1. It separates public-facing components from internal services.
2. It ensures that the database, Redis, and Kafka are not directly exposed to users.
3. It places authentication and authorization checks inside the backend API.

4. It supports audit logging for important actions such as login attempts, product creation, stock updates, deletion, and unauthorized access attempts.
5. It supports monitoring and alerting for suspicious behavior such as repeated failed logins and unusual inventory activity.
6. It supports Docker and Kubernetes deployment because each major service can run as an independent container or pod.
7. It supports Zero Trust because every request must be verified before access is granted.
8. It supports future security testing because authentication, authorization, validation, logging, and monitoring points are clearly defined.

5. Component Architecture

The Secure Inventory Management Platform is divided into multiple components so that each part has a clear responsibility and can be secured separately. The main components are the user layer, entry layer, application layer, database layer, cache layer, event streaming layer, and monitoring layer.

Component	Purpose	Security Role
External Users	Admin, Manager, Staff/User, and Auditor access the platform through browser.	Users must authenticate before accessing protected features.
Ingress Controller / Reverse Proxy	Receives HTTPS requests and forwards them to the application.	Applies TLS, routing, rate limiting, and secure traffic handling.
Secure Inventory Application	Node.js and Express backend that handles main business logic.	Performs authentication, RBAC, validation, audit logging, and secure API handling.
Authentication Module	Handles login, password verification, JWT/session handling, and logout.	Prevents unauthorized access and account spoofing.
RBAC Module	Checks user role and permission before sensitive operations.	Prevents privilege escalation and unauthorized actions.

Inventory Module	Handles product creation, stock updates, stock viewing, and item deletion.	Protects inventory integrity through validation, authorization, and logging.
Supplier and Order Module	Handles supplier details and purchase/order records.	Protects business records from unauthorized access or modification.
Reporting Module	Generates inventory, supplier, stock, and audit reports.	Restricts reports based on role and prevents report abuse.
Audit Logging Module	Records login attempts, stock changes, product changes, role changes, and sensitive actions.	Provides accountability and supports investigation.
MongoDB Database	Stores users, inventory records, suppliers, orders, audit logs, and security alerts.	Kept inside private network and accessed only by backend API.
Redis	Stores temporary security data such as failed-login counters and cache values.	Supports brute-force detection and temporary security state handling.
Kafka Broker	Publishes inventory and security events.	Supports event-driven monitoring and security visibility.
Monitoring and Alerts	Collects logs, metrics, suspicious activity, and system health data.	Helps detect attacks, failures, and abnormal behavior.

The normal flow starts when a user sends an HTTPS request. The request reaches the Ingress Controller or Reverse Proxy and is forwarded to the Secure Inventory Application. The application authenticates the user, checks role permissions, validates input, performs the requested inventory operation, and then communicates with MongoDB, Redis, or Kafka as required. Important actions are logged and sent to the monitoring layer for detection and response.

6. Data Flow and Trust Boundaries

The Secure Inventory Management Platform uses controlled data flow between users, application services, database, cache, event streaming, and monitoring components. The architecture separates public-facing traffic from internal services so that sensitive components such as MongoDB, Redis, and Kafka are not directly exposed to external users.

6.1 Data Flow

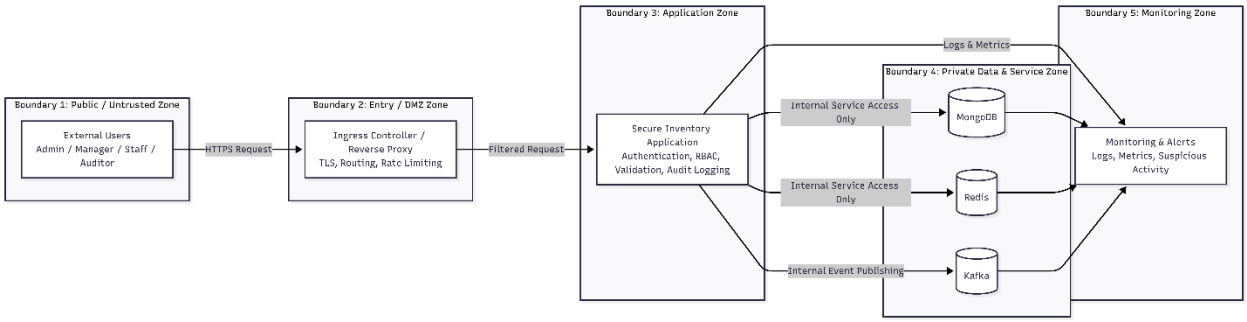
Step	Data Flow	Description	Security Control
1	**User Browser → Ingress / Reverse Proxy**	User sends login or inventory request through HTTPS.	TLS, secure headers, rate limiting
2	**Ingress / Reverse Proxy → Application**	Valid request is forwarded to the Node.js/Express application.	Controlled routing and request filtering
3	**Application → Authentication Module**	User credentials or JWT/session token are verified.	Password hashing, JWT validation, session expiry
4	**Application → RBAC Module**	User role and permission are checked.	Backend authorization and least privilege
5	**Application → MongoDB**	Inventory, user, supplier, order, audit, and security data are stored or retrieved.	Private network access, validation, secrets
6	**Application → Redis**	Failed-login counters and temporary security states are stored.	Internal access only and limited data lifetime
7	**Application → Kafka**	Inventory and security events are published.	Event-driven logging and activity tracking
8	**Application / Services → Monitoring**	Logs, metrics, alerts, and suspicious activity are collected.	Detection, alerting, and incident support

6.2 Trust Boundaries

Trust Boundary	Components Inside	Risk Controlled
Public Network Boundary	External users and browser traffic	Controls untrusted internet/user access
Entry Boundary	Ingress Controller / Reverse Proxy	Filters traffic before it reaches the application

Application Boundary	Node.js/Express application and security modules	Enforces authentication, RBAC, validation, and audit logging
Private Cluster Boundary	MongoDB, Redis, Kafka	Prevents direct external access to internal services
Monitoring Boundary	Logs, metrics, alerts, and security events	Supports detection, investigation, and response

Trust Boundary Diagram



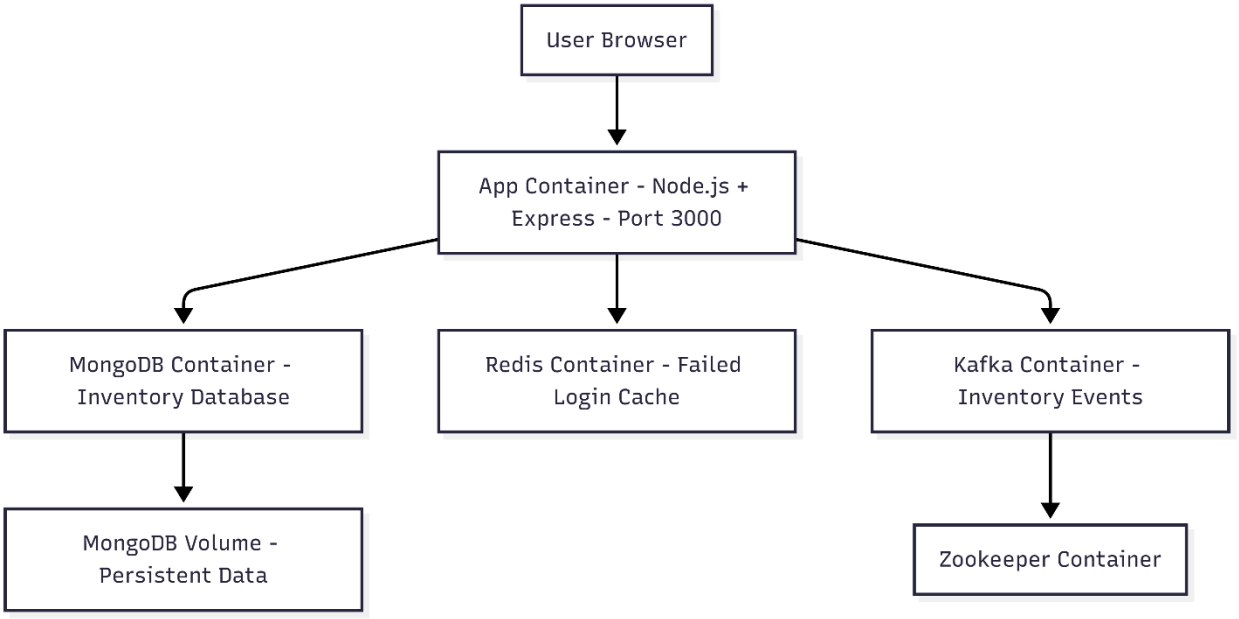
The most important trust boundary is between the public user network and the internal application environment. External users can only reach the system through HTTPS and the Ingress Controller or Reverse Proxy. They cannot directly access MongoDB, Redis, or Kafka.

The backend application acts as the main security enforcement point. It verifies user identity, checks role permissions, validates input, performs business operations, records audit logs, and sends important events to monitor. Internal services remain protected inside the private cluster boundary.

7. Docker Architecture

The Secure Inventory Management Platform uses Docker to package the application and its supporting services into isolated containers. Docker makes the system easier to deploy, test, and run consistently across different environments.

Diagram



7.2 Docker Components

Docker Component	Purpose	Security Benefit
App Container	Runs the Node.js and Express inventory application.	Isolates application runtime from the host system.
MongoDB Container	Stores users, inventory records, suppliers, orders, audit logs, and security alerts.	Keeps database separate from application code.
Redis Container	Stores temporary security state such as failed-login counters and cache values.	Supports brute-force detection and temporary security tracking.
Zookeeper Container	Supports Kafka coordination.	Keeps Kafka support service separate.
Kafka Container	Handles inventory and security event publishing.	Supports event-driven logging and monitoring.
MongoDB Volume	Stores MongoDB data persistently.	Prevents data loss when the database container restarts.

7.3 Docker Security Design

The application container is designed to run with production settings and a minimal Node.js Alpine image. Only production dependencies are installed, which reduces unnecessary packages in the final container. The Dockerfile also creates a non-root application user and runs the application using that user instead of root.

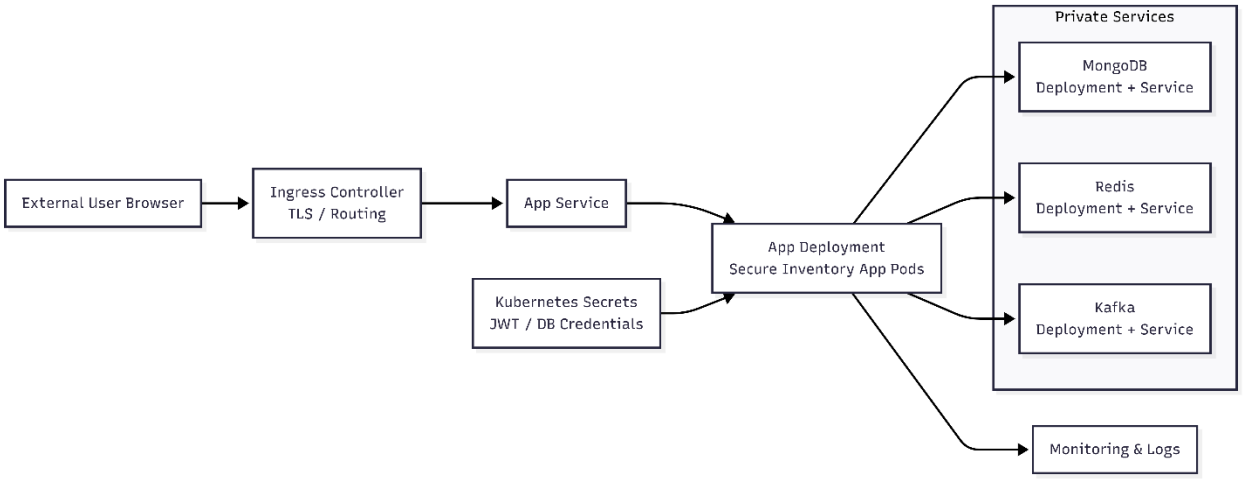
The Docker Compose architecture separates the application, database, Redis, Kafka, and Zookeeper into different containers. This separation limits the impact of failure or compromise in one service. Environment variables are used to pass configuration such as database connection strings, Redis URL, Kafka broker address, and JWT secret.

For stronger production security, default secrets should be replaced with strong values, internal services should not be exposed publicly, images should be scanned before deployment, and containers should run with least privilege.

8. Kubernetes Architecture

Kubernetes is used to deploy and manage the Secure Inventory Management Platform in a container orchestration environment. The Kubernetes architecture separates the application, database, cache, event streaming, secrets, services, and network controls into different resources. This improves scalability, availability, and security.

Diagram



8.2 Kubernetes Components

Kubernetes Component	Purpose	Security Benefit

Ingress Controller	Receives external traffic and routes it to the application service.	Provides controlled entry point, TLS support, and request routing.
App Service	Exposes the application pods inside the cluster.	Avoids direct access to individual pods.
App Deployment	Runs the Secure Inventory Management application pods.	Supports replicas, restart, scaling, and controlled rollout.
MongoDB Deployment/Service	Runs and exposes MongoDB internally.	Keeps database inside the cluster network.
Redis Deployment/Service	Runs Redis for failed-login tracking and cache data.	Keeps temporary security state internal.
Kafka Deployment/Service	Runs Kafka for inventory and security events.	Supports internal event-driven communication.
Kubernetes Secrets	Stores sensitive values such as JWT secret and database credentials.	Avoids hardcoding secrets in source code or manifests.
Network Policy	Restricts communication between pods and services.	Reduces unnecessary internal access.
Monitoring and Logs	Collects pod logs, metrics, and suspicious activity.	Supports detection and incident response.

8.3 Kubernetes Security Design

In this architecture, external users do not directly access application pods, MongoDB, Redis, or Kafka. Traffic first reaches the Ingress Controller, then moves to the App Service, and finally reaches the application pods. Internal services such as MongoDB, Redis, and Kafka are only accessed by the backend application.

Kubernetes Secrets are used for sensitive configuration values instead of storing secrets directly in code. Network Policies restrict unnecessary pod-to-pod communication. The application deployment should run with non-root containers, resource limits, health checks, and least-privilege access. These controls help reduce the impact of container compromise and improve platform reliability.

9. Zero Trust Architecture

The Secure Inventory Management Platform follows the Zero Trust principle: never trust automatically, always verify. In this architecture, no user, request, service, container, or internal component is trusted by default. Every access request must be authenticated, authorized, validated, logged, and monitored.

9.1 Zero Trust Design Principles

Zero Trust Principle	Application in Secure Inventory Management Platform
Verify every user	Every user must log in before accessing protected inventory features.
Enforce least privilege	Admin, Manager, Staff/User, and Auditor only receive permissions required for their role.
Validate every request	Backend APIs validate JWT/session tokens, user roles, and request input.
Do not trust frontend controls	Sensitive permissions are checked on the backend, not only by hiding frontend buttons.
Protect internal services	MongoDB, Redis, and Kafka are private services and are not directly exposed to external users.
Secure secrets	JWT secrets and database credentials are stored using environment variables or Kubernetes Secrets.
Monitor continuously	Login attempts, failed logins, inventory changes, suspicious activity, and system logs are monitored.
Log sensitive actions	Stock updates, product deletion, role changes, login attempts, and security events are recorded.

9.2 Zero Trust Flow

When a user sends a request, the system does not trust the request automatically. The request first passes through the Ingress Controller or Reverse Proxy. The backend then verifies the user identity using authentication controls. After authentication, the RBAC module checks whether the user has permission to perform the requested action.

If the request is valid, the application processes it and communicates with internal services such as MongoDB, Redis, or Kafka. These services are not directly available to external users. Every sensitive action is logged and monitored for suspicious behavior.

9.3 Zero Trust Benefits

The Zero Trust design reduces the risk of account misuse, privilege escalation, unauthorized stock modification, database exposure, and insider abuse. It also improves accountability because important actions are logged and monitored continuously.

10. SABSA Architecture Mapping

SABSA is used to connect business security needs with the technical architecture of the Secure Inventory Management Platform. It helps show how business requirements, security objectives, logical design, physical deployment, and operational controls are connected.

SABSA Layer	Focus	Application in Secure Inventory Management Platform
Contextual Architecture	Business security requirements	The platform must protect inventory records, supplier data, order records, user accounts, roles, and audit logs from unauthorized access or modification.
Conceptual Architecture	Security principles and objectives	The system follows confidentiality, integrity, availability, accountability, least privilege, Zero Trust, and secure-by-design principles.
Logical Architecture	Security functions and relationships	Authentication, RBAC, input validation, audit logging, monitoring, Redis failed-login tracking, Kafka event publishing, and database protection are defined as logical security functions.
Physical Architecture	Technology and deployment design	The platform uses Node.js/Express, MongoDB, Redis, Kafka, Docker containers, Kubernetes services, secrets, deployments, and network policies.
Component Architecture	Actual system components	App container, MongoDB container/service, Redis container/service, Kafka service, Kubernetes Secrets, Ingress Controller, monitoring, and logs are the main components.
Operational Architecture	Runtime security and management	Logs, metrics, failed-login monitoring, suspicious activity detection, health checks, resource limits, backups, and alerts support secure operations.

11. Security Controls Mapping

This section maps the main security controls of the Secure Inventory Management Platform to the architecture components where they are applied.

Security Control	Applied Component	Purpose
HTTPS / TLS	Browser, Ingress Controller, Application	Protects data in transit from sniffing and interception.
Authentication	Secure Inventory Application	Ensures only valid users can access protected features.
JWT / Session Validation	Backend API	Verifies user identity on protected requests.
Role-Based Access Control	Backend API, RBAC Module	Ensures users can only perform actions allowed by their role.
Least Privilege	Application, Kubernetes, Database	Reduces unnecessary access to sensitive functions and services.
Input Validation	Backend API, Inventory Module, Supplier Module	Prevents malicious or invalid input from reaching business logic or database.
Password Hashing	Authentication Module, Database	Protects stored user passwords.
Rate Limiting	Ingress Controller, Backend API	Reduces brute-force attempts and API abuse.
Audit Logging	Application, Audit Logging Module, MongoDB	Records important actions for accountability and investigation.
Failed Login Tracking	Redis	Detects repeated failed login attempts.
Event Publishing	Kafka	Sends inventory and security events for monitoring and analysis.
Kubernetes Secrets	Kubernetes Cluster	Stores sensitive values such as JWT secrets and database credentials.

Network Policy	Kubernetes Internal Services	Restricts unnecessary pod-to-pod communication.
Monitoring and Alerts	Monitoring Layer	Detects suspicious activity, errors, and abnormal system behavior.
Persistent Storage	MongoDB Volume / Database Storage	Preserves inventory and audit data after restart or failure.
Container Isolation	Docker Containers, Kubernetes Pods	Separates application, database, Redis, and Kafka services.
Non-root Container Execution	Application Container	Reduces the impact of container compromise.
Resource Limits	Kubernetes Deployment	Prevents excessive resource usage and supports availability.
Health Checks	Kubernetes Deployment	Detects unhealthy pods and supports automatic recovery.
Backup and Recovery	Database Layer	Supports restoration after failure, corruption, or attack.

These controls help protect the platform against spoofing, tampering, repudiation, information disclosure, denial of service, and privilege escalation. The controls are placed across the entry layer, application layer, data layer, container layer, Kubernetes layer, and monitoring layer.

12. Deployment View

The deployment view explains how the Secure Inventory Management Platform is arranged during execution. The system can run locally using Docker Compose and can also be deployed in Kubernetes for orchestration, scaling, and secure service management.

Deployment Layer	**Component**	**Description**
Client Layer	User Browser	Admin, Manager, Staff/User, and Auditor access the platform through a browser.
Entry Layer	Ingress Controller / Reverse Proxy	Receives external HTTPS traffic and forwards requests to the application service.

Application Layer	Secure Inventory App	Runs the Node.js and Express backend application inside a container or Kubernetes pod.
Service Layer	App Service	Provides stable internal access to application pods in Kubernetes.
Data Layer	MongoDB	Stores users, inventory records, suppliers, orders, audit logs, and security alerts.
Cache Layer	Redis	Stores temporary security state such as failed-login counters and cache data.
Event Layer	Kafka and Zookeeper	Kafka publishes inventory and security events, while Zookeeper supports Kafka coordination.
Secret Layer	Kubernetes Secrets / Environment Variables	Stores sensitive configuration such as JWT secrets and database credentials.
Monitoring Layer	Monitoring and Logs	Collects logs, metrics, alerts, and suspicious activity information.
Storage Layer	MongoDB Volume / Persistent Storage	Preserves database records after container or pod restart.

In Docker deployment, the application, MongoDB, Redis, Kafka, and Zookeeper run as separate containers. This helps isolate services and makes the system easier to run locally.

In Kubernetes deployment, the application runs as pods managed by a deployment. The app is exposed internally through a service and externally through an Ingress Controller. MongoDB, Redis, and Kafka are deployed as internal services. Kubernetes Secrets are used for sensitive values, and monitoring collects runtime logs and security-related events.

13. Architecture Risk Considerations

This section identifies important architecture-level risks that may still exist in the Secure Inventory Management Platform and explains how the architecture reduces them.

Risk	Possible Impact	Architecture Mitigation

Weak authentication	Attacker may access the system using stolen or guessed credentials.	Secure login, password hashing, JWT/session validation, failed-login tracking, and rate limiting.
Broken access control	Staff/User may access Admin or Manager functions.	Backend RBAC, least privilege, and server-side authorization checks.
Public exposure of database	Attacker may directly access inventory, supplier, order, or audit data.	MongoDB is placed inside the private internal service layer and accessed only by the application.
API abuse or brute force	Login and inventory APIs may be overloaded or abused.	Ingress rate limiting, backend rate limiting, Redis failed-login tracking, and monitoring alerts.
Input tampering	Malicious input may corrupt records or trigger injection attacks.	Input validation, safe query handling, schema validation, and backend checks.
Secret leakage	JWT secrets or database credentials may be exposed.	Kubernetes Secrets and environment variables are used instead of hardcoding sensitive values.
Container compromise	Attacker may abuse container privileges.	Non-root container execution, container isolation, resource limits, and minimal images.
Internal service misuse	Compromised service may access unnecessary internal components.	Network policies and private service access reduce unnecessary communication.
Loss of audit evidence	Users may deny sensitive actions such as stock updates or deletions.	Audit logging records important actions with user, time, and action details.
Service failure	Application or database failure may affect availability.	Kubernetes deployments, health checks, restart support, persistent storage, and backup planning.

The architecture reduces these risks by applying layered security controls across the entry layer, application layer, private services, container environment, Kubernetes resources, and monitoring layer

Conclusion:

Phase 2 explains the secure architecture blueprint for the Secure Inventory Management Platform. The design explains how users, the application, MongoDB, Redis, Kafka, Docker, Kubernetes, secrets, and monitoring components interact securely. The architecture applies authentication, RBAC, least privilege, input validation, audit logging, and private internal service access. Docker supports isolated container-based deployment, while Kubernetes supports orchestration, scaling, secrets, health checks, and network control. Zero Trust principles are applied by verifying every user, validating every request, protecting internal services, and monitoring sensitive actions. SABSA mapping connects the technical architecture with business goals such as inventory integrity, accountability, confidentiality, and availability. Overall, this architecture provides a secure foundation for the next phases: secure implementation, testing, Docker hardening, and Kubernetes deployment.